VR Mini Project 1

Multi-Object Apparel Detection and Instance Segmentation

Project Repository:

`https://github.com/KhSudhir2345/VR_MiniProject_1`

KH Sudhir	IMT2023546
Abhinav Kishan	IMT2023580
Nori Sai Sandeep	IMT2023574
Lithin Sai Kumar	IMT2023598

---

0 Contents

1 Task 3.1 — Classification **2**
1.1 Problem Description . . . 2
1.2 Dataset Description . . . 2
1.3 Data Preprocessing . . . 4
1.4 Model Architectures . . . 5
1.5 Training Strategies . . . 6
1.6 Results — Task 3.1 Classification . . . 6
1.6.1 ResNet50 — Transfer Learning . . . 6
1.6.2 ResNet50 — Training from Scratch . . . 7
1.6.3 EfficientNet-B0 — Transfer Learning . . . 8
1.6.4 EfficientNet-B0 — Training from Scratch . . . 9
1.6.5 MobileNetV2 — Transfer Learning . . . 10
1.6.6 MobileNetV2 — Training from Scratch . . . 12
1.6.7 Summary Comparison — All Models . . . 14

2 Task 3.2 — Segmentation **14**
2.1 Problem Description . . . 14
2.2 Dataset Description — Segmentation . . . 15
2.3 Data Preprocessing — Mask R-CNN . . . 16
2.4 Data Preprocessing — U-Net . . . 17
2.5 Data Preprocessing — YOLO . . . 18
2.6 Model Architecture — Mask R-CNN . . . 19
2.7 Model Architecture — U-Net . . . 20
2.8 Model Architecture — YOLO . . . 20
2.9 Training Details . . . 21
2.10 Training Details — YOLO . . . 22
2.11 Evaluation Metrics . . . 23

3 Results and Analysis **24**
3.1 Mask R-CNN Results . . . 24
3.2 U-Net Results . . . 26
3.3 YOLO Results . . . 27
3.4 Comparative Analysis . . . 30

1 Task 3.1 — Classification

1.1 Problem Description

The objective of this task is to develop a deep learning-based image classification model that can accurately classify clothing items into predefined categories. The dataset used is a subset of **DeepFashion2**, focusing on five major clothing categories:

- Short Sleeve Top
- Long Sleeve Top
- Shorts
- Trousers
- Skirt

Each input image may contain one or more clothing items; the goal is to assign the most appropriate class label based on the dominant clothing item present. The key challenges include:

- Different poses and orientations of clothing items
- Occlusions and multiple objects in a single image
- Variability in lighting, background, and scale

1.1.0 Objectives

- Preprocess and prepare the dataset for training
- Design and train a classification model capable of distinguishing between the clothing categories
- Evaluate the model using standard metrics: accuracy, precision, recall, and F1-score
- Analyse the model's performance and identify areas for improvement

1.2 Dataset Description

1.2.0 Dataset Overview

DeepFashion2 is a large-scale, diverse dataset designed for fashion understanding tasks such as detection, segmentation, and classification. It contains images collected from real-world scenarios including online shopping platforms and street photography, making it highly challenging due to variations in pose, scale, occlusion, and background.

1.2.0 Dataset Structure

The dataset is organised into three standard splits:

- Train Set

- Validation Set
- Test Set

Each image is associated with a JSON annotation file containing: clothing category labels, bounding boxes, segmentation polygons, and landmark annotations. For this classification task, only the **category labels** were utilised.

1.2.0 Selected Classes

Five major clothing classes were selected for this project:

1. Short Sleeve Top
2. Long Sleeve Top
3. Shorts
4. Trousers
5. Skirt

These classes are common in real-world fashion datasets, visually distinguishable, and representative of both upper and lower body clothing.

1.2.0 Dataset Size

Table 1: Dataset split sizes used for training

Split	**Approximate Size**
Training Images	∼72,000 (50% of dataset)
Validation Images	∼23,000

1.2.0 Data Characteristics

Table 2: Challenging properties of the dataset

Property	**Description**
Multi-object images	Some images contain multiple clothing items
Occlusion	Clothing items may be partially hidden
Scale variation	Objects appear at different sizes
Background clutter	Complex backgrounds may affect classification
Pose variation	Clothing worn by models in different poses

1.2.0 Class Distribution

The dataset is not perfectly balanced across all classes. Some categories (e.g. tops or trousers) appear more frequently than others (e.g. skirts or shorts). This class imbalance must be considered during training and evaluation.

1.3 Data Preprocessing

1.3.0 Image Filtering and Selection

The original dataset was filtered to retain only images containing at least one instance of the five selected categories. Images without relevant categories were discarded.

1.3.0 Multi-Label Encoding

Since a single image may contain multiple clothing items, the task was formulated as a **multi-label classification problem**. Each image was assigned a *multi-hot encoded* label vector, where each position represents the presence (1) or absence (0) of a specific class.

Table 3: Class-to-index mapping

Index	Class
0	Short Sleeve Top
1	Long Sleeve Top
2	Shorts
3	Trousers
4	Skirt

Example label vector: `[1, 0, 0, 1, 0]` indicates the presence of *Short Sleeve Top* and *Trousers*.

1.3.0 Image Resizing

All images were resized to a fixed resolution of 224×224 pixels before being passed to the model, ensuring uniformity and efficient batch processing.

1.3.0 Train–Validation Split

Table 4: Final train/validation split

Set	Approximate Number of Images
Training	∼72,000
Validation	∼23,000

1.4 Model Architectures

1.4.0 EfficientNet-B0

EfficientNet is a family of convolutional neural networks that scales model dimensions — depth, width, and resolution — in a balanced manner using a *compound scaling* method. Key features include:

- MBConv (Mobile Inverted Bottleneck Convolution) blocks
- Squeeze-and-Excitation (SE) modules for channel attention
- High accuracy with relatively fewer parameters

Implementation: EfficientNet-B0 was used; the final classification layer was replaced with a fully connected layer, and a **sigmoid activation** function was applied for multi-label prediction.

1.4.0 MobileNetV2

A lightweight CNN designed for efficiency on resource-constrained devices. Key features:

- Depthwise separable convolutions to reduce computation
- Inverted residual blocks with linear bottlenecks
- Optimised for speed and low memory usage

Implementation: MobileNetV2 pretrained on ImageNet; the final layer was modified and sigmoid activation applied.

1.4.0 ResNet

A deep CNN that addresses the vanishing-gradient problem via *residual (skip) connections*. Key features:

- Skip connections allowing gradients to flow directly
- Enables training of very deep networks without degradation
- Learns residual mappings instead of direct mappings

Implementation: ResNet pretrained on ImageNet; the fully connected output layer was modified and sigmoid activation applied.

1.5 Training Strategies

Table 5: Comparison of training strategies

Strategy	Description	Notes
Transfer Learning	Pretrained ImageNet weights; initial layers optionally frozen; final layer retrained	Faster convergence; better performance with limited data
Training from Scratch	All weights randomly initialised; entire network trained on dataset	Slower convergence; lower performance; requires more data

1.5.0 Loss Function

Binary Cross-Entropy with Logits Loss (BCEWithLogitsLoss) was used, treating each class independently — ideal for multi-label classification.

1.5.0 Evaluation Metrics

- Precision
- Recall
- F1-score (Macro and Micro)
- ROC-AUC

Macro F1-score was considered the primary metric as it treats all classes equally regardless of class imbalance.

1.6 Results — Task 3.1 Classification

1.6.1 ResNet50 — Transfer Learning

Table 6: Classification results — ResNet50 (Transfer Learning)

Class	Precision	Recall	F1
Short Sleeve Top	0.9088	0.8825	0.8955
Trousers	0.9171	0.9179	0.9175
Shorts	0.8410	0.8316	0.8363
Long Sleeve Top	0.8332	0.8071	0.8200
Skirt	0.9033	0.8392	0.8701
Macro Average	**0.8807**	**0.8557**	**0.8679**
Micro Average	**0.8910**	**0.8669**	**0.8788**

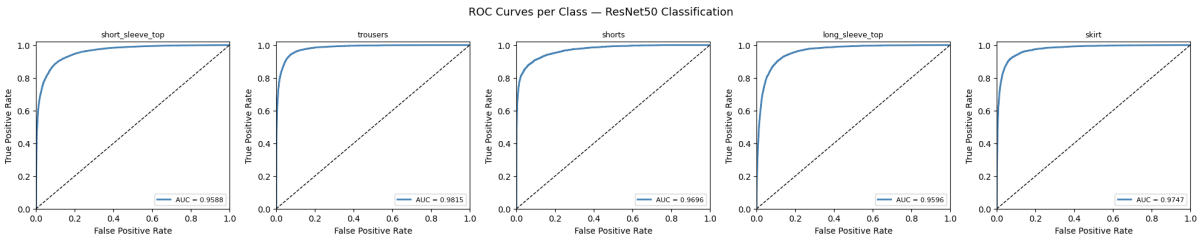
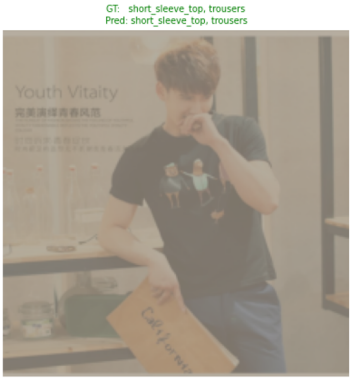


Figure 1: ROC Curves for ResNet (Transfer Learning)

(a) Short Sleeve Top



(b) Short Sleeve Top

Figure 2: Sample Predictions — ResNet50 (Transfer Learning)

1.6.2 ResNet50 — Training from Scratch

Table 7: Classification results — ResNet50 (Training from Scratch)

Class	**Precision**	**Recall**	**F1**
Short Sleeve Top	0.8582	0.8240	0.8407
Trousers	0.8996	0.8354	0.8663
Shorts	0.8317	0.6789	0.7476
Long Sleeve Top	0.7324	0.6275	0.6759
Skirt	0.7637	0.7958	0.7794
Macro Average	**0.8171**	**0.7523**	**0.7820**
Micro Average	**0.8303**	**0.7762**	**0.8023**

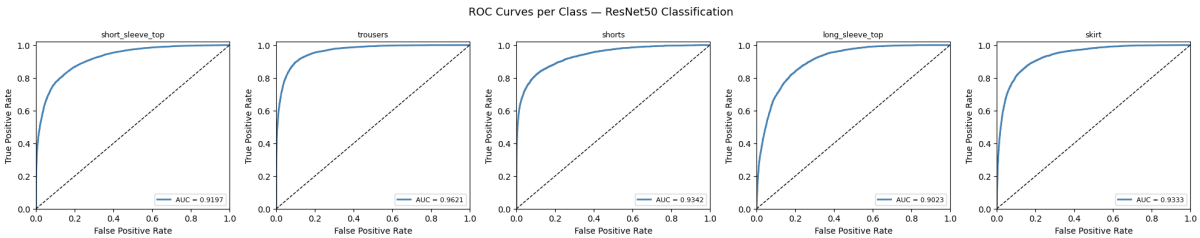
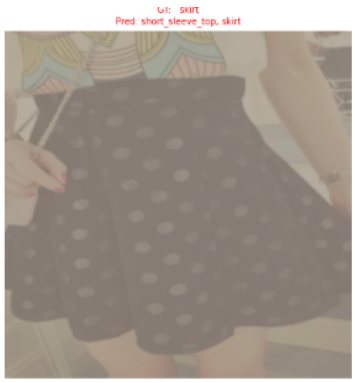


Figure 3: ROC Curves for ResNet (From Scratch)



(a) Skirt

(b) short sleeve top

Figure 4: Sample Predictions — ResNet50 (Transfer Learning)

1.6.3 EfficientNet-B0 — Transfer Learning

Table 8: Classification results — EfficientNet-B0 (Transfer Learning)

Class	Precision	Recall	F1
Short Sleeve Top	0.8954	0.8699	0.8824
Trousers	0.8998	0.9106	0.9051
Shorts	0.8354	0.7907	0.8124
Long Sleeve Top	0.8069	0.7770	0.7917
Skirt	0.8840	0.8160	0.8486
Macro Average	**0.8643**	**0.8328**	**0.8481**
Micro Average	**0.8748**	**0.8480**	**0.8612**

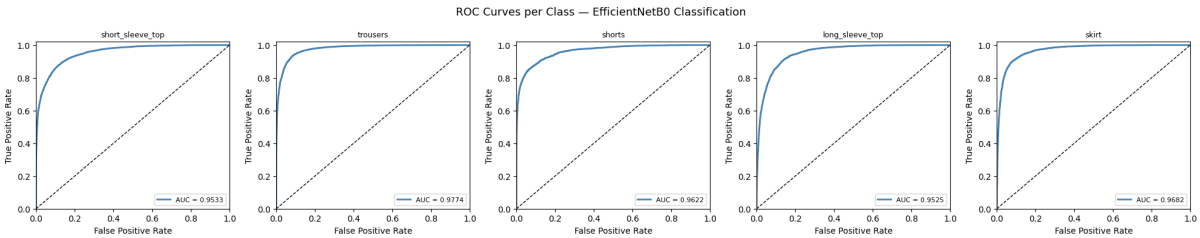
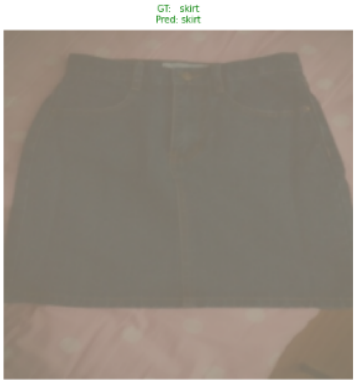


Figure 5: ROC Curves for EfficientNet(Transfer Learning)



(a) skirt

(b) Short Sleeve Top

Figure 6: Sample Predictions — EfficientB0 (Transfer Learning)

1.6.4 EfficientNet-B0 — Training from Scratch

Table 9: Classification results — EfficientNet-B0 (Training from Scratch)

Class	Precision	Recall	F1
Short Sleeve Top	0.8133	0.7773	0.7949
Trousers	0.8471	0.8394	0.8432
Shorts	0.6776	0.6337	0.6549
Long Sleeve Top	0.6614	0.5642	0.6090
Skirt	0.7685	0.5915	0.6685
Macro Average	**0.7536**	**0.6812**	**0.7141**
Micro Average	**0.7793**	**0.7129**	**0.7446**

(a) skirt

(b) trousers

Figure 7: Sample Predictions — EfficientB0 (Scratch)

1.6.5 MobileNetV2 — Transfer Learning

Table 10: Classification results — MobileNetV2 (Transfer Learning)

Class	Precision	Recall	F1
Short Sleeve Top	0.8882	0.8381	0.8624
Trousers	0.8938	0.8833	0.8885
Shorts	0.8049	0.7498	0.7764
Long Sleeve Top	0.7622	0.7676	0.7649
Skirt	0.8526	0.7974	0.8240
Macro Average	**0.8403**	**0.8072**	**0.8233**
Micro Average	**0.8548**	**0.8220**	**0.8381**

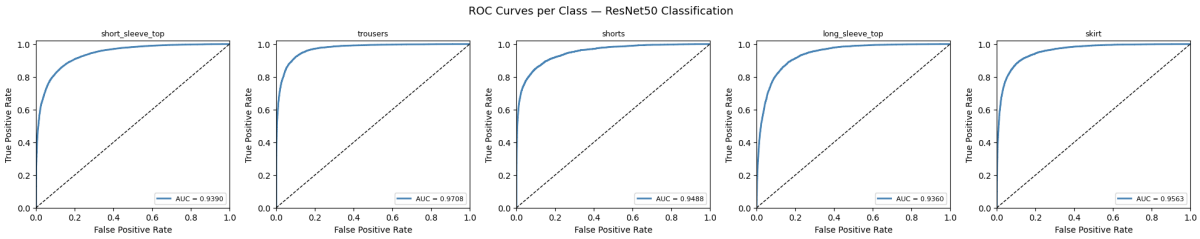


Figure 8: ROC Curves for EfficientNet(Transfer Learning)

(a) trousers

(b) Short Sleeve Top

Figure 9: Sample Predictions — MobileNet (Transfer)

1.6.6 MobileNetV2 — Training from Scratch

Table 11: Classification results — MobileNetV2 (Training from Scratch)

Class	Precision	Recall	F1
Short Sleeve Top	0.7425	0.8031	0.7716
Trousers	0.8346	0.7917	0.8126
Shorts	0.6207	0.6046	0.6126
Long Sleeve Top	0.6338	0.4708	0.5403
Skirt	0.7310	0.5164	0.6052
Macro Average	**0.7125**	**0.6373**	**0.6685**
Micro Average	**0.7371**	**0.6792**	**0.7070**

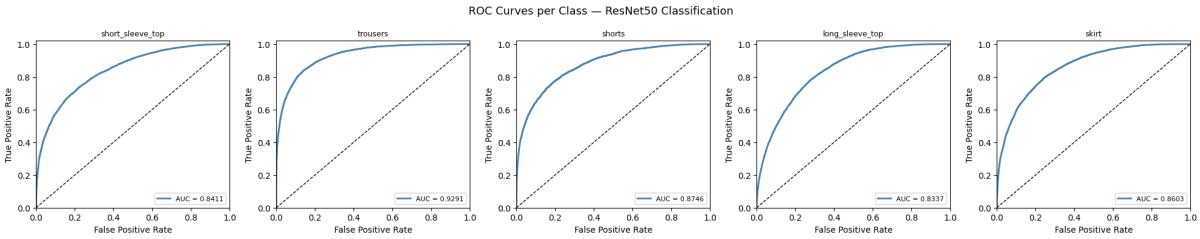
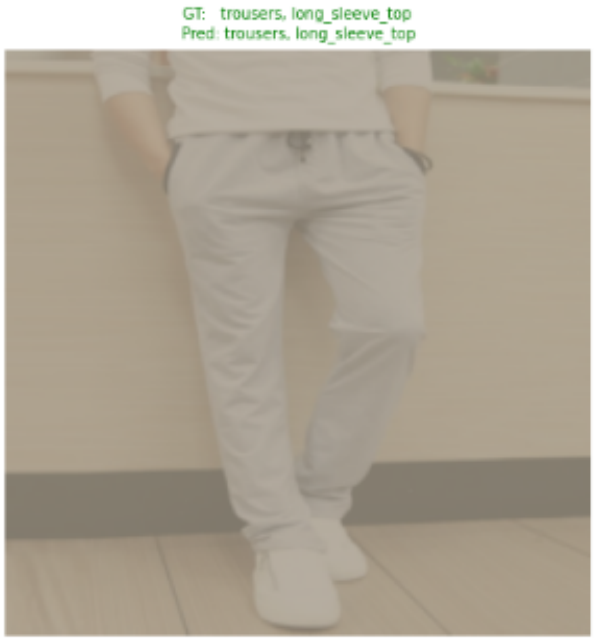
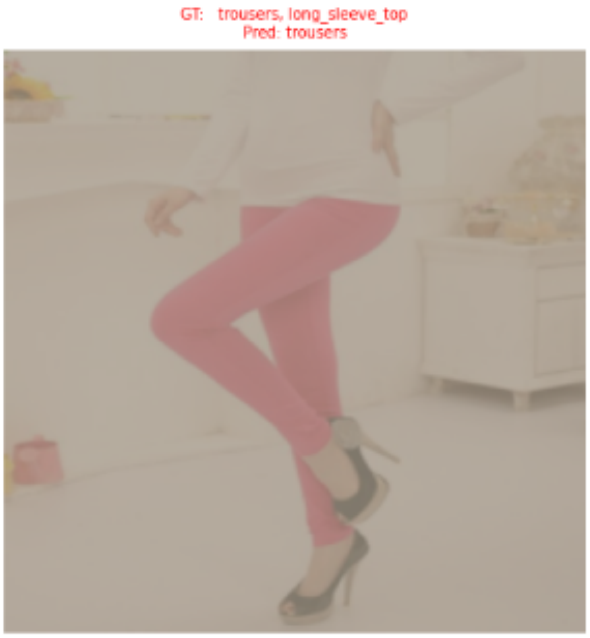


Figure 10: ROC Curves for EfficientNet(Transfer Learning)



(a) trousers



(b) Short Sleeve Top

Figure 11: Sample Predictions — MobileNet (Scratch)

1.6.7 Summary Comparison — All Models

Table 12: Summary of macro-averaged metrics across all models and strategies

Model	Strategy	Macro P	Macro R	Macro F1
ResNet50	Transfer Learning	0.8807	0.8557	**0.8679**
ResNet50	From Scratch	0.8171	0.7523	0.7820
EfficientNet-B0	Transfer Learning	0.8643	0.8328	0.8481
EfficientNet-B0	From Scratch	0.7536	0.6812	0.7141
MobileNetV2	Transfer Learning	0.8403	0.8072	0.8233
MobileNetV2	From Scratch	0.7125	0.6373	0.6685

2 Task 3.2 — Segmentation

2.1 Problem Description

The objective of this task is to develop a model capable of performing **clothing segmentation**, where each pixel in an image is classified into one of the predefined clothing categories.

The segmentation task focuses on the same five clothing categories used in classification:

- Short Sleeve Top
- Long Sleeve Top
- Shorts
- Trousers
- Skirt

Each image may contain multiple clothing items, and the model must correctly identify and segment each instance at the pixel level.

2.1.0 Type of Segmentation

Table 13: Segmentation paradigms used in this project

Type	Description	Model
Instance Segmentation	Each object detected and segmented individually; outputs bounding boxes and segmentation masks	YOLOv8-seg
Semantic Segmentation	Each pixel assigned a class label; multiple objects of the same class treated as a single region	U-Net

2.1.0 Objectives

- Train a model capable of accurately segmenting clothing items
- Generate pixel-level masks for each class
- Evaluate model performance using:
  - mAP (mean Average Precision — for detection and segmentation)
  - mIoU (mean Intersection over Union)
  - Dice Score

2.2 Dataset Description — Segmentation

The dataset used is again **DeepFashion2**. Unlike classification tasks, this dataset provides detailed **pixel-level annotations** in the form of segmentation polygons, making it suitable for both semantic and instance segmentation.

2.2.0 Annotation Details

Each clothing item is annotated using polygon coordinates defining the object boundary, in the form:

```
[class_id  x1  y1  x2  y2  ...  xn  yn]
```

These coordinates are converted into:

- **Normalised YOLO segmentation format** (for YOLOv8-seg)
- **Pixel-wise masks** (for U-Net)

2.2.0 Data Preparation for Models

Table 14: Data preparation pipeline per model

Model	Annotation Conversion	Output Format
YOLOv8-seg	Polygon annotations converted directly into YOLO segmentation format	Normalised polygon coordinates
U-Net	Polygons converted into pixel-level masks; class-wise segmentation maps generated	Pixel-wise mask images

2.3 Data Preprocessing — Mask R-CNN

For Mask R-CNN, the dataset was preprocessed to convert raw annotations into instance-level masks, bounding boxes, and labels.

2.3.0 Annotation Parsing and Filtering

JSON annotations were parsed to extract:

- Category ID
- Bounding boxes
- Segmentation polygons

Only the selected five clothing classes were retained and mapped to label indices ($0 \rightarrow 4$).

2.3.0 Mask Generation

Polygon annotations were converted into binary masks:

- Each object instance $\rightarrow$ one mask
- Pixel value: 1 (object), 0 (background)

2.3.0 Instance Handling

For each image, multiple objects were stored as:

- Bounding boxes
- Class labels
- Corresponding masks

2.3.0 Data Formatting

Data was structured in the format required by Mask R-CNN:

- Image tensor
- Target dictionary with `boxes`, `labels`, `masks`

2.3.0 Transformations

- Image resizing
- Normalization (ImageNet)
- Optional augmentations (e.g., horizontal flip)

2.4 Data Preprocessing — U-Net

For U-Net, preprocessing focused on generating pixel-level segmentation maps from polygon annotations.

2.4.0 Annotation Parsing and Filtering

JSON annotations were parsed to extract segmentation polygons and category IDs. Only the selected five classes were retained and mapped to label indices ($0 \rightarrow 4$).

2.4.0 Mask Generation

Polygon annotations were converted into pixel-wise masks:

- All instances of the same class merged into one mask
- Each pixel assigned a class label

2.4.0 Semantic Map Creation

For each image, a single segmentation map was created where:

- Each pixel stores its class index
- Background assigned a separate label

2.4.0 Data Formatting

Each sample consists of:

- Input image
- Corresponding segmentation mask

2.4.0 Transformations

- Image and mask resizing to fixed resolution
- Normalization of images
- Optional augmentations (flip, rotation)

2.5 Data Preprocessing — YOLO

For YOLO-based detection and segmentation, the dataset was converted into the required YOLO format using annotation parsing and normalization.

2.5.0 Annotation Parsing and Filtering

JSON annotation files were processed to extract:

- Category IDs
- Bounding boxes
- Segmentation polygons

Only the selected five clothing classes were retained and mapped to label indices ($0 \rightarrow 4$).

2.5.0 YOLO Format Conversion

Annotations were converted into YOLO format:

- **Detection**:
$$(class_id,\ x_{center},\ y_{center},\ width,\ height)$$
- **Segmentation**: Normalized polygon coordinates:
$$(class_id,\ x_1,\ y_1,\ x_2,\ y_2,\ \ldots,\ x_n,\ y_n)$$

All coordinates were normalized with respect to image dimensions.

2.5.0 Handling Multiple Objects

Each image may contain multiple clothing items:

- Each object stored as a separate annotation
- All annotations saved in corresponding label files

2.5.0 Data Organization

The dataset was structured in YOLO format:

- `images/train`, `images/val`
- `labels/train`, `labels/val`

Each image has a corresponding label file with the same name.

2.5.0 Image Preprocessing

- Images resized to a fixed resolution
- Normalization applied during training
- Augmentations such as flipping and scaling used

2.6 Model Architecture — Mask R-CNN

Mask R-CNN is an instance segmentation model that extends Faster R-CNN by adding a parallel branch for predicting segmentation masks.

2.6.0 Backbone

A CNN backbone (e.g., ResNet-50) with Feature Pyramid Network (FPN) is used to extract multi-scale feature maps.

2.6.0 Region Proposal Network (RPN)

The RPN generates candidate object regions (proposals) by predicting:

- Objectness scores
- Bounding box coordinates

2.6.0 ROI Heads

For each proposed region:

- **Classification Head**: predicts class labels
- **Bounding Box Head**: refines box coordinates
- **Mask Head**: predicts a binary segmentation mask

2.6.0 Output

The model outputs, for each detected instance:

- Bounding box
- Class label
- Pixel-level mask

2.7 Model Architecture — U-Net

U-Net is a semantic segmentation architecture designed to perform pixel-wise classification using an encoder–decoder structure.

2.7.0 Encoder

The encoder consists of convolutional and pooling layers that:

- Extract hierarchical features
- Reduce spatial resolution

2.7.0 Decoder

The decoder progressively upsamples feature maps to recover spatial details:

- Uses transposed convolutions / upsampling
- Combines features via skip connections

2.7.0 Skip Connections

Feature maps from the encoder are concatenated with decoder layers to:

- Preserve fine-grained details
- Improve localization accuracy

2.7.0 Output

The model produces a segmentation map where:

- Each pixel is assigned a class label

2.8 Model Architecture — YOLO

YOLO (You Only Look Once) is a one-stage object detection model that performs detection and segmentation in a single forward pass.

2.8.0 Backbone

A convolutional neural network backbone is used to extract hierarchical features from the input image:

- Captures low-level to high-level features
- Efficient and optimized for real-time performance

2.8.0 Neck

The neck (Feature Pyramid Network / PAN) aggregates multi-scale features:

- Combines features from different layers
- Improves detection of objects at varying scales

2.8.0 Detection Head

The detection head predicts for each object:

- Bounding box coordinates
- Objectness score
- Class probabilities

2.8.0 Segmentation Head

For segmentation, YOLO additionally predicts:

- Prototype masks (shared across objects)
- Mask coefficients for each instance

These are combined to generate instance-level segmentation masks.

2.8.0 Output

For each detected object, the model outputs:

- Bounding box
- Class label
- Segmentation mask

2.9 Training Details

Both Mask R-CNN and U-Net models were trained using a consistent training setup to ensure fair comparison.

2.9.0 Training Strategy

- Transfer learning using pretrained backbones (ImageNet)
- Fine-tuning of all layers after initial training

2.9.0 Mask R-CNN Training

- Backbone: ResNet-50 with FPN
- Optimizer: Stochastic Gradient Descent (SGD)
- Loss components:
  - Classification loss
  - Bounding box regression loss
  - Mask loss (binary cross-entropy)

2.9.0 U-Net Training

- Encoder–decoder trained end-to-end
- Optimizer: Adam
- Loss function:
  - Cross-entropy loss (pixel-wise)
  - Dice loss (to handle class imbalance)

2.9.0 Common Settings

- Batch size chosen based on GPU memory constraints
- Learning rate scheduling applied
- Early stopping used to prevent overfitting

2.10 Training Details — YOLO

The YOLO model was trained for joint object detection and instance segmentation using a unified training pipeline.

2.10.0 Training Strategy

- Transfer learning using pretrained weights (COCO)
- Fine-tuning on the DeepFashion2 subset

2.10.0 Optimization

- Optimizer: Stochastic Gradient Descent (SGD)
- Learning rate scheduling applied during training
- Batch size selected based on GPU memory constraints

2.10.0 Loss Function

The total loss is a combination of multiple components:

- **Bounding Box Loss**: measures localization error
- **Classification Loss**: measures class prediction accuracy
- **Objectness Loss**: predicts presence of objects
- **Segmentation Loss**: evaluates mask prediction quality

2.10.0 Data Augmentation

To improve generalization, the following augmentations were applied:

- Random horizontal flipping
- Scaling and resizing
- Color jittering

2.10.0 Training Setup

- Training performed on GPU-enabled environment
- Early stopping used to prevent overfitting
- Model checkpoints saved based on validation performance

2.11 Evaluation Metrics

Model performance was evaluated using standard metrics for both detection and segmentation tasks.

2.11.0 Mask R-CNN Metrics

- **mAP@[0.5:0.95]**: Evaluates detection and instance segmentation performance across IoU thresholds
- **Mean Intersection over Union (mIoU)**: Measures overlap between predicted and ground truth masks

2.11.0 U-Net Metrics

- **Mean Intersection over Union (mIoU)**: Measures pixel-level overlap accuracy
- **Dice Score**: Evaluates similarity between predicted and ground truth masks

2.11.0 Metric Definitions

Intersection over Union:

$$IoU = \frac{TP}{TP + FP + FN}$$

Dice Score:

$$Dice = \frac{2TP}{2TP + FP + FN}$$

2.11.0 Evaluation Strategy

- Metrics computed per class
- Macro-averaging used for overall performance

3 Results and Analysis

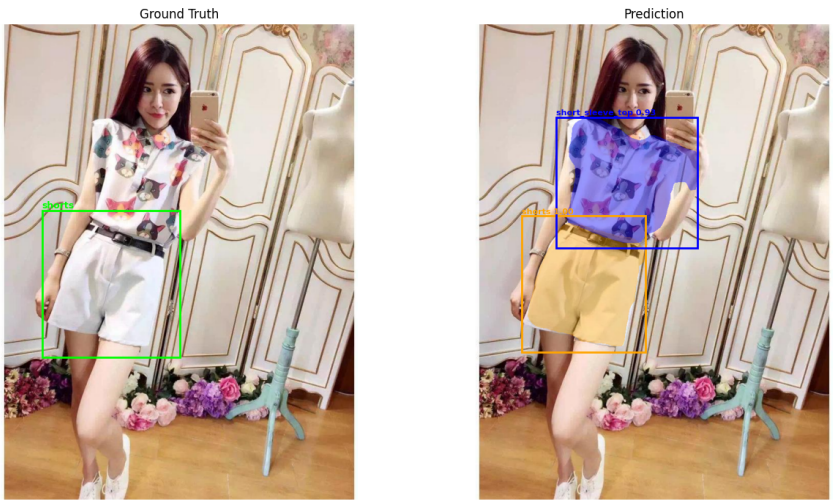
This section presents the performance of Mask R-CNN and U-Net models under both transfer learning and training from scratch. Evaluation is performed using detection metrics (mAP) and segmentation metrics (mIoU and Dice Score).

3.1 Mask R-CNN Results

3.1.0 Training from Scratch

Table 15: Mask R-CNN (Training from Scratch) Performance

Metric	Value
mAP@[0.5:0.95]	0.6325
mIoU	0.85
Dice Score	0.9151
Macro F1-score	0.7076
Micro F1-score	0.7306



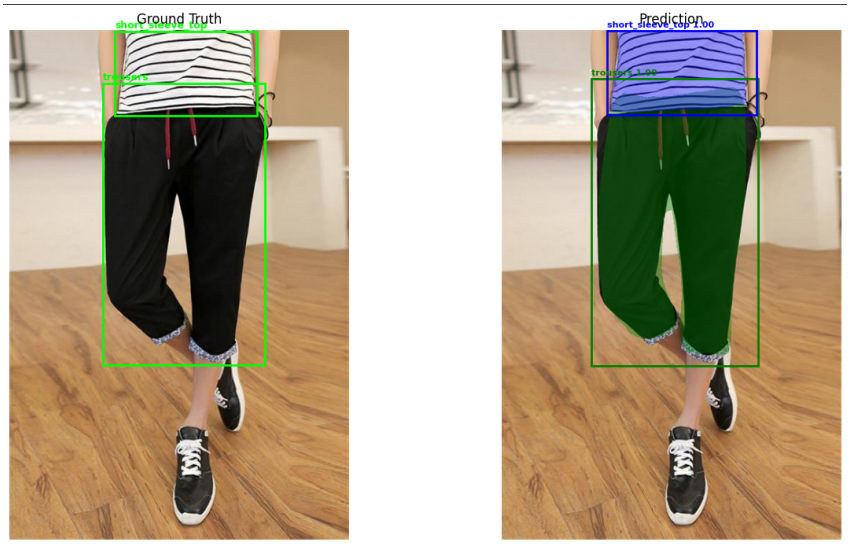
(a) Short Sleeve Top

Figure 12: Sample Predictions — MaskRCNN (Scratch)

3.1.0 Transfer Learning

Table 16: Mask R-CNN (Transfer Learning) Performance

Metric	Value
mAP@[0.5:0.95]	0.69
mIoU	0.86
Dice Score	0.9218
Macro F1-score	0.7
Micro F1-score	0.72



(a) trousers

Figure 13: Sample Predictions — MaskRCNN (Transfer)

3.1.0 Observations

- Mask R-CNN performs well for instance-level segmentation.
- mAP reflects strong detection capability, while mIoU and Dice capture segmentation quality.
- Transfer learning improves both detection and mask accuracy.

3.2 U-Net Results

3.2.0 Transfer Learning

Table 17: U-Net (Fine Tuning) Performance

Metric	Value
mAP@[0.5:0.95]	0.3978
mIoU	0.6369
Dice Score	0.7774
Macro F1-score	0.7655

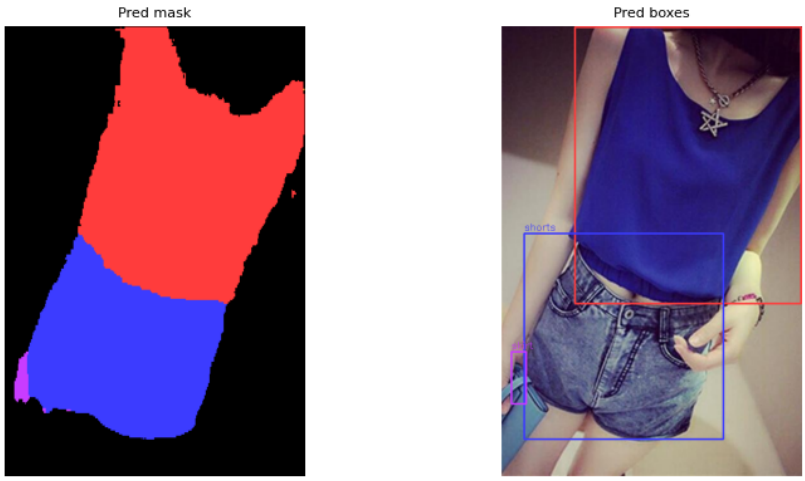
(a) Short Sleeve Top

Figure 14: Sample Predictions — U-Net (Transfer)

3.2.0 Training from Scratch

Table 18: U-Net (Training from Scratch) Performance

Metric	Value
mAP@[0.5:0.95]	0.2634
mIoU	0.4766
Dice Score	0.6409
Macro F1-score	0.5535



(a) Short Sleeve Top

Figure 15: Sample Predictions — U-Net (Transfer)

3.2.0 Observations

- U-Net achieves strong pixel-level segmentation performance.
- Dice score indicates robustness to class imbalance.
- Transfer learning improves convergence and segmentation quality.

3.2.0 Key Insights

- Mask R-CNN excels in instance-level detection and segmentation.
- U-Net provides better dense pixel-wise segmentation accuracy.
- Transfer learning significantly improves performance for both models.
- There is a trade-off between detection capability (Mask R-CNN) and fine-grained segmentation (U-Net).

3.3 YOLO Results

3.3.0 Training from Scratch

Table 19: YOLO (Training from Scratch) Performance

Metric	Value
mAP@0.5	0.54
mAP@[0.5:0.95]	0.44
mIoU	0.4641
dice	0.5101

(a) Detection Output

Figure 16: Sample Predictions — YOLO (Scratch)

3.3.0 Transfer Learning

Table 20: YOLO (Transfer Learning) Performance

Metric	Value
mAP@0.5	0.66
mAP@[0.5:0.95]	0.57
mIoU	0.46
dice$_{s}core$	0.51

(a) Detection Output

Figure 17: Sample Predictions — YOLO (Transfer)

3.3.0 Observations

- YOLO demonstrates strong real-time detection capability with efficient inference speed.
- Transfer learning significantly improves mAP, mIoU, and convergence speed.
- Training from scratch requires more epochs and shows comparatively lower performance.
- YOLO performs well in object localization but may be slightly less precise than Mask R-CNN for segmentation tasks.

3.4 Comparative Analysis

Table 21: Overall Comparison of Models

Model	**mIoU**	**Dice Score**
Mask R-CNN (Transfer)	0.85	0.91
Mask R-CNN (Scratch)	0.86	0.9218
U-Net (Transfer)	0.63	0.77
U-Net (Scratch)	0.47	0.64